



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

EICTA: First Project Delivery - ER and SQL

Author(s): **David Loredan Barb (10664390)**

Letizia Castellani (11016807)

Federica Stefanizzi (10727743)

Rocco Tonon (10740668)

Isabella Zamboni (10929976)

Group Number: **1**

Academic Year: 2023-2024

Contents

Contents	i
1 INTRODUCTION	1
2 TEXT ANALYSIS	3
3 ER	5
3.1 Entity-Relationship Model	5
3.1.1 Entities	6
3.1.2 Relationships	7
4 LOGICAL MODEL	9
4.1 Entities	9
4.2 Relationships	9
4.3 Description of Logical Model	10
5 RELATIONAL MODEL	11
5.1 Relational Schema Explanation	12
6 SQL	13
6.1 SQL1 - Building the Database	13
6.1.1 Creating the Database	13
6.1.2 Create Tables	13
6.2 SQL2 - Populating the DataBase	19
6.2.1 Mockaroo Example	19
6.2.2 Python Script: Random Values Generator	20
6.2.3 Table Data Snapshots	21
6.3 SQL3 - Performing the Queries	31
6.3.1 "WHERE" QUERIES	31
6.3.2 "WHERE, LIMIT, LIKE" QUERIES	33

6.3.3	"WHERE, IN, Nested Query" QUERIES	36
6.3.4	"GROUP BY, 1 JOIN, AS" QUERIES	38
6.3.5	"WHERE, GROUP BY" QUERIES	40
6.3.6	"GROUP BY, HAVING, AS" QUERIES	41
6.3.7	"WHERE, GROUP BY, HAVING, AS" QUERIES	43
6.3.8	"WHERE, Nested Query, GROUP BY" QUERIES	46
6.3.9	"WHERE, GROUP BY, HAVING, 1 JOIN" QUERIES	47
6.3.10	"WHERE, GROUP BY, HAVING, 2 JOINS" QUERIES	49

7	CONCLUSION	51
----------	-------------------	-----------

1 | INTRODUCTION

The primary goal of this project is to resolve the indicated challenge relative to the creation of a data design for a music application. The solution that is suggested takes a methodical approach, beginning with the creation of an Entity-Relationship (ER) model. The ER model provides an abstract and conceptual view of the database structure, representing all the entities and their connections inside the system.

Following that, a logical model is developed, with the aim of depicting the entities and their connections in greater detail. This phase tries to provide an effective structure of database design, resulting in a more refined and structured representation than the abstract ER model.

A relational model was created to translate the conceptual and logical representations into a set of tables with defined attributes and to show how these attributes connect the different tables through the use of primary and foreign keys, in order to support the structure provided by the logical model with some visual representation.

Finally, MySQL was utilized for the database's practical creation and execution. The full database conforms to the relational model's initial requirements and integrates them into ten separate tables. To conclude, a set of sample queries are conducted to demonstrate the seamless functioning of the entire database; they are performed mainly to serve as practical demonstrations of the ability of the created data structure to store and retrieve relevant information effectively.

2 | TEXT ANALYSIS

The First Project Delivery refers to the following text:

*“Consider the problem of specifying the data design for a music application. **USERS** enrol into the platform by providing unique **mail**, **username**, and **password**. Their **registration date** is stored, too. Users can be **premium** users if they pay a monthly fee. Users **listen to** **PODCASTS** and **SONGS**. Songs **are grouped** into **CATEGORIES**, described by a **name** and a **description**, and are played by various **artists** while various authors produce podcasts. Podcasts are described by a unique **identifier**, **title**, **duration** (in seconds), and **publication date**. Songs include all the attributes of podcasts, including the **text** of the song. The app **tracks who listens to what, when, and whether they listened to the whole song or podcast**. Users can create **PLAYLISTS** that **aggregate** songs. These are described by a **unique identifier**, **title**, **creation date**, and whether they are **public**. **The current date is also stored whenever a song is added to a playlist**. Users can **also tag a song or a playlist as a favourite**. The current date is also stored whenever a song or a playlist is tagged as a favourite.”*

Within the text provided, **five** separate **entities** have been recognised, each clearly displayed in uppercase and distinguished by distinctive colour choices. These entities include users, podcasts, songs, categories, and playlists, each with specific attributes that define them. Furthermore, an additional **sixth entity**, identified as Audio_Content, has been established as a "parent" entity to both the Podcast and Song entities. This conceptual connection is shown in using a consistent blue colour for both the entities and features related to Podcast and Song.

The defined **relationships** between all the entities are emphasized in bold across the text, explaining the data structure's numerous interactions. Users, who are crucial to the entire scheme, can interact with both songs and podcasts, and these connections enable the database to carefully track user preferences, thus determining which content between Song and Podcast is accessed, when, and whether it has been listened to in its whole. In addition to listening, users are empowered to create playlists.

Songs, in turn, can be both categorised into different categories and aggregated into playlists. This aggregation creates a database structure that allows the music application to record the date a song was added to a playlist.

Finally, **two new relations** have been defined: one between a User and a Playlist, and another between a User and a Song, with the specific objective of allowing users to mark songs or playlists as favourites, and of storing the date in which this preference is expressed.

An Entity-Relation scheme was modelled by using the software "draw.io" and by following the requirements of the text's First Project Delivery:



3.1.1. Entities

The **first identified entity** is the one representing the User who is using the music application. Users are recognised by a unique EMAIL, a RegistrationDate, a Username, a Password, and an attribute called IsPremium, which is assumed to be a boolean variable indicating whether or not a given user pays a monthly fee.

The **second entity** that has been identified is Audio_Content, which is the parent of a **Total and Exclusive ISA hierarchy**. The hierarchy is considered to be Total because it is comprised of only two child entities, Song and Podcast. Furthermore, it can be described as Exclusive since a given instance of audio content cannot be both a Podcast and a Song at the same time, but only one of them. Audio_Content is identified by a unique CONTENT_ID, a Title, a Duration, and a PublicationDate, which are attributes shared by both Song and Podcast. Nevertheless, those two child entities present only their individual attributes, which were not inherited by the parent. Song is further identified by Text and Artist, whilst Podcast includes a column disclosing the authors. It is critical to note that, due to design choices and assumptions, the Artist who wrote a Song instance and the Author of a Podcast are represented as attributes of their respective entities, rather than as entities themselves. Finally, a significant assumption is that a Song may have been produced by more than one Artist and a Podcast by more than one Author: if this is the case, the names of those who participated in the song's writing or the podcast production will be presented in a single row in a unique string.

Another entity identified in the text is Category, which is assumed to be defined by a unique NAME as its primary key, and a Description. For a more intuitive understanding, each Song is presumed to belong to exactly one category

The last identified entity is Playlist, which is recognised by a distinctive PLAYLIST_ID, a Title, a CreationDate, and an attribute called IsPublic, which is presumed to be a boolean variable indicating whether or not a playlist may be accessed by everyone.

3.1.2. Relationships

The **first identified relation** is Listens to, which links *User* and *Audio_Content*. It is composed of two attributes called When and Whole, which respectively indicate when a User listened to audio content and whether it was listened to in its entirety. A User may listen to an unlimited amount of *Audio_Content*, whether or not it is a Song or Podcast, and the same *Audio_Content* may be listened to by an unlimited number of users. According to the When attribute's date-time format, a User can listen to a single *Audio_Content* multiple times in a single day or over several days, preventing the presence of duplicates in the database (check the dump of the DataBase in order to see the 'When' attribute as the primary key of type DATETIME).

Song and *Category* are **connected by a relation** that defines whether a Song is part of a particular Category. While a Category can contain indefinite instances of Song, a song is assumed and required to be part of exactly one Category.

Song and *Playlist* are connected via an **aggregative relation** that is solidified by the InsertionDate attribute, which provides the precise day on which a Song was inserted into a Playlist. Realistically, in order for a playlist to be qualified as such, it must contain at least one song and, while a Playlist is assumed to be created by exactly one User, each User is allowed to create an indefinite number of playlists. A peculiar assumption of this database aims to be sure that, in order to prevent duplicates, a Song can be aggregated into a Playlist only once.

Additionally, the Music Application is required to store whenever a *Song* or *Playlist* is tagged as favourite by a *User*, and this is the rationale behind the creation of the **two relations** 'TagsPlaylist' and 'TagsSong', as they both include the attribute TagDate indicating when a Playlist or a Song has been marked as favourite. During the establishment process of these relations, several assumptions have been taken care of. While each User can tag indefinite Song or Playlist instances as favourites, and those instances can be tagged as favourites by an indefinite number of users, a user is actually unable to mark the same Song or Playlist as a favourite more than once, even if tagged in different days; nevertheless, different songs and playlists can be marked as favourites on the same day.

4 | LOGICAL MODEL

4.1. Entities

- *User*(**EMAIL**, Username, Password, RegistrationDate, IsPremium)
- *Category*(**NAME**, Description)
- *Song*(**CONTENT_ID**, Title, Duration, PublicationDate, Artist, Text, NAME)
- *Podcast*(**CONTENT_ID**, Title, Duration, PublicationDate, Author)
- *Playlist*(**PLAYLIST_ID**, Title, CreationDate, IsPublic, EMAIL)

4.2. Relationships

- *UserListensToSong*(**EMAIL**, **CONTENT_ID**, **WHEN**, Whole)
- *UserListensToPodcast*(**EMAIL**, **CONTENT_ID**, **WHEN**, Whole)
- *Song_Playlist*(**CONTENT_ID**, **PLAYLIST_ID**, InsertionDate)
- *User_Song*(**EMAIL**, **CONTENT_ID**, TagDate)
- *User_Playlist*(**EMAIL**, **PLAYLIST_ID**, TagDate)

4.3. Description of Logical Model

In this phase of the work, the ER model has been translated to the **Logical Model**. All entities have been initially defined, and the model has been handled differently depending on the type of connection. More specifically, in the case of a "1-N" connection, the Primary Key of the "1" entity has been incorporated into the "N" side as a Foreign Key. This is the case of the "Relationship Aggregate" that takes place between *User* and *Playlist*, in which the Primary Key "*Email*" in *User* is also a Foreign Key in *Playlist*.

Subsequently, with regard to the "N-N" relationships, they have been represented in the model through bridge tables. These tables uniquely encapsulate the combination of primary keys from the entities they connect. Essentially, the primary keys of a bridge table serve as composite keys, representing the coupling of the primary keys of the associated entities. In the case of peculiar instances, such as in *UserListensToSong* and *UserListensToPodcast*, the attribute 'When' has also been designated as a primary key to prevent duplications of users listening to the same song or podcast multiple times, even in the same day or hour.

Regarding the "Listens to" relationship that connects *User* and *Audio Content*, it has been assumed the existence of two specific relationships: one between *User* and *Song* and the other one between *User* and *Podcast*. This was undertaken to facilitate MySQL in distinguishing the origin of the foreign key, whether from the 'Song' or 'Podcast,' as it was impractical to establish a single relationship connecting 'User' to both 'Song' and 'Podcast'.

5 | RELATIONAL MODEL

A Relational Model was designed using the software "draw.io":

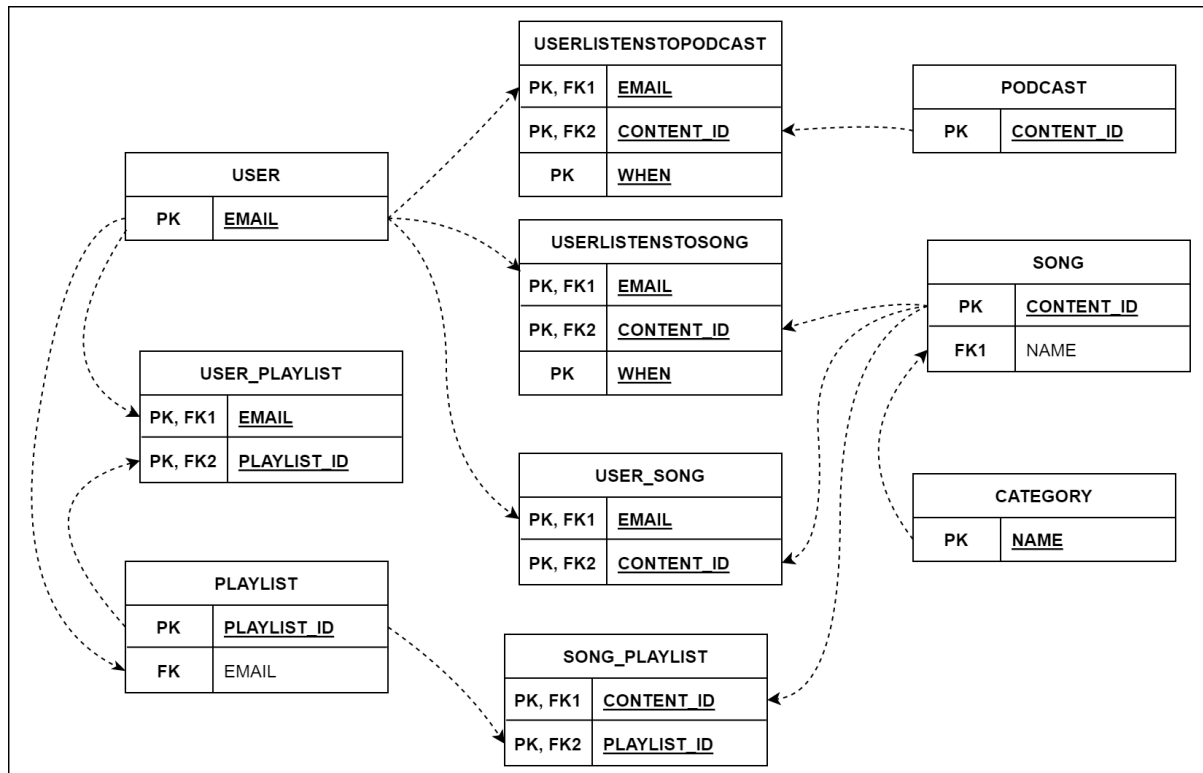


Figure 5.1: Relational Model

5.1. Relational Schema Explanation

The relational model maintains consistency with the logical model's assumptions, which are now more prominently represented through the use of tables displaying only the **Primary** and **Foreign Keys**. In particular, in this phase, the links between the primary and foreign keys of the different tables have been emphasised. This is valid specifically for the bridge tables that are connected to the attributes of the linked specific table.

Relationships 1 to N or N to 1 are shown as single links between two tables; for example, a single user can create one or multiple playlists, while a playlist is created by one and only one user (the same holds for the link between Song and Category entities).

This model highlights the Foreign Key Constraints when creating the tables, so that SQL doesn't allow the insertion of random values in bridge tables that are not present in the Primary Keys of the tables linked to them, which is a practice that can violate the ER model design.

6 | SQL

6.1. SQL1 - Building the Database

The initial part covers the implementation and elucidation of the "musicapp" DB, which is provided as a self-contained dump accompanying this document (check for "DumpMusicApp.sql" file).

6.1.1. Creating the Database

```
CREATE DATABASE `MusicApp`;  
USE `MusicApp`;
```

These two lines create the Database called 'MusicApp' and use it in order to be able to create new tables in it.

6.1.2. Create Tables

Common conventions are implemented while creating the tables, such as the fact that all attributes are NOT NULL except for attributes that have a TEXT data type; another thing is that all attributes VARCHAR are set to a common maximum 64 characters (if it is too short then it can be set to 128, 256, or 512). Another convention is used for all attributes that are IDs, which is by using SMALLINT UNSIGNED. Thus, an ID is a non-negative integer of 2 bytes (if too short it can be changed to an INT UNSIGNED type). Check the next page for the implementation of the 10 tables in the Database.

User Table

```
CREATE TABLE `User`(  
    `Email` VARCHAR(64) PRIMARY KEY,  
    `Username` VARCHAR(64) NOT NULL,  
    `Password` VARCHAR(64) NOT NULL,  
    `RegistrationDate` DATE NOT NULL,  
    `IsPremium` BOOLEAN NOT NULL  
);
```

An assumption is made for premium subscribers (IsPremium attribute), which is by using a boolean value to check if a user has a premium account or not.

Category Table

```
CREATE TABLE `Category`(  
    `Name` VARCHAR(64) PRIMARY KEY,  
    `Description` TEXT  
);
```

As mentioned in subsection 6.1.2, 'Description' attribute is assumed to the possibility of being NULL since it is of TEXT type and it's not necessary to have a description for each category.

Song Table

```
CREATE TABLE `Song`(  
  `Content_ID` SMALLINT UNSIGNED PRIMARY KEY,  
  `Title` TINYTEXT NOT NULL,  
  `Duration` TIME NOT NULL,  
  `PublicationDate` DATE NOT NULL,  
  `Artist` VARCHAR(64) NOT NULL,  
  `Text` TEXT,  
  `Name` VARCHAR(64),  
  CONSTRAINT `Song_fk` FOREIGN KEY (`Name`) REFERENCES `Category` (`Name`)  
);
```

Here 'Title' must not be null since a song is supposed to have a title. Same as for Category table, 'Text' attribute can be NULL (some songs are instrumental, thus a text doesn't exist). Constraint Foreign Key of Category 'Name' attribute is got from relationship 1-to-N. In fact, a song is expected to be part of only one category while multiple different songs can be part of a single category. (Check section 3.1.2 - Relationships)

Podcast Table

```
CREATE TABLE `Podcast`(  
  `Content_ID` SMALLINT UNSIGNED PRIMARY KEY,  
  `Title` TINYTEXT NOT NULL,  
  `Duration` TIME NOT NULL,  
  `PublicationDate` DATE NOT NULL,  
  `Author` VARCHAR(64) NOT NULL  
);
```

Almost everything similar to 'Song' table. In fact, both are children of 'Audio_Content' entity (ISA Relationship).

Playlist Table

```
CREATE TABLE `Playlist`(  
  `Playlist_ID` SMALLINT UNSIGNED PRIMARY KEY,  
  `Title` TINYTEXT NOT NULL,  
  `CreationDate` DATE NOT NULL,  
  `IsPublic` BOOLEAN NOT NULL,  
  `Email` VARCHAR(64),  
  CONSTRAINT `Playlist_fk` FOREIGN KEY (`Email`) REFERENCES `User` (`Email`)  
);
```

Here there is another Constraint Foreign Key of User 'Email' attribute from the 1-to-N or N-to-1 relationship. It is all explained in the ER Model section. (check section 3.1.2 - Relationships)

UserListensToSong Table

```
CREATE TABLE `UserListensToSong`(  
  `Email` VARCHAR(64),  
  `Content_ID` SMALLINT UNSIGNED,  
  `When` DATETIME NOT NULL,  
  `Whole` BOOLEAN NOT NULL,  
  
  PRIMARY KEY (`Email`, `Content_ID`, `When`),  
  CONSTRAINT `UserListensToSong_fk_1` FOREIGN KEY (`Email`) REFERENCES `User` (`Email`),  
  CONSTRAINT `UserListensToSong_fk_2` FOREIGN KEY (`Content_ID`) REFERENCES `Song` (`Content_ID`)  
);
```

This is the first bridge table. In fact, there are two Constraints of Foreign Keys that take the primary keys of linked entities' tables (one of 'User' and one of 'Song'). As mentioned in section 4.3 - "Description of Logical Model", 'When' attribute is added as primary key in order to let the Music App memorizing in the Database a single user listening to the same song multiple times a day (it is assumed that a user cannot listen to the same song multiple times at the same date and time instant, it makes sense).

UserListensToPodcast Table

```
CREATE TABLE `UserListensToPodcast`(  
  `Email` VARCHAR(64),  
  `Content_ID` SMALLINT UNSIGNED,  
  `When` DATETIME NOT NULL,  
  `Whole` BOOLEAN NOT NULL,  
  
  PRIMARY KEY (`Email`, `Content_ID`, `When`),  
  CONSTRAINT `UserListensToPodcast_fk_1` FOREIGN KEY (`Email`) REFERENCES `User` (`Email`),  
  CONSTRAINT `UserListensToPodcast_fk_2` FOREIGN KEY (`Content_ID`) REFERENCES `Podcast` (`Content_ID`)  
);
```

This table is very similar to 'UserListensToSong'. The key concepts are the same.

Song_Playlist Table

```
CREATE TABLE `Song_Playlist`(  
  `Content_ID` SMALLINT UNSIGNED,  
  `Playlist_ID` SMALLINT UNSIGNED,  
  `InsertionDate` DATE NOT NULL,  
  
  PRIMARY KEY (`Content_ID`, `Playlist_ID`),  
  CONSTRAINT `Song_Playlist_fk_1` FOREIGN KEY (`Content_ID`) REFERENCES `Song` (`Content_ID`),  
  CONSTRAINT `Song_Playlist_fk_2` FOREIGN KEY (`Playlist_ID`) REFERENCES `Playlist` (`Playlist_ID`)  
);
```

This bridge table has a particular taking: playlists cannot have the same song multiple times. Thus, only foreign keys are also primary keys of 'Song_Playlist'.

User_Song Table

```
CREATE TABLE `User_Song`(  
  `Email` VARCHAR(64),  
  `Content_ID` SMALLINT UNSIGNED,  
  `TagDate` DATE NOT NULL,  
  
  PRIMARY KEY (`Email`, `Content_ID`),  
  CONSTRAINT `User_Song_fk_1` FOREIGN KEY (`Email`) REFERENCES `User` (`Email`),  
  CONSTRAINT `User_Song_fk_2` FOREIGN KEY (`Content_ID`) REFERENCES `Song` (`Content_ID`)  
);
```

Same idea of 'Song_Playlist' table, only foreign keys as primary keys. In fact, a user is not supposed to tag the same song more than once.

User_Playlist Table

```
CREATE TABLE `User_Playlist`(  
  `Email` VARCHAR(64),  
  `Playlist_ID` SMALLINT UNSIGNED,  
  `TagDate` DATE NOT NULL,  
  
  PRIMARY KEY (`Email`, `Playlist_ID`),  
  CONSTRAINT `User_Playlist_fk_1` FOREIGN KEY (`Email`) REFERENCES `User` (`Email`),  
  CONSTRAINT `User_Playlist_fk_2` FOREIGN KEY (`Playlist_ID`) REFERENCES `Playlist` (`Playlist_ID`)  
);
```

This table closely resembles 'User_Song' table.

6.2. SQL2 - Populating the DataBase

Tables are filled with random insertion values that respect the different data types and constraints. To attain this level of randomness, tools such as "mockaroo" (link: <https://www.mockaroo.com/>) and Python scripts were employed. Songs, podcasts, and categories were added manually to be more realistic, but they could be added through random generation as others.

6.2.1. Mockaroo Example

An example of screenshots on how to generate random values for 'User' table:

Field Name	Type	Options
Email	Email Address	blank: 0 %
Username	Username	blank: 0 %
Password	Password	minimum length: 14 upper: 1 lower: 1 numbers: 1 symbols: 1 blank: 0 %
RegistrationDate	Datetime	11/09/2022 to 11/09/2023 format: yyyy-mm-dd blank: 0 %
IsPremium	Boolean	blank: 0 %

+ ADD ANOTHER FIELD GENERATE FIELDS USING AI...

Rows: 25 Format: SQL Table Name: MOCK_DATA ☐ include CREATE TABLE

Figure 6.1: Field's Setting of User Table

Email	Username	Password	RegistrationDate	IsPremium
ahartmann0@slate.com	jcristofanini0	iA2 (fhCh)0K\$WyRm\$fZLC?Y	2023-06-01	false
bminney1@youtu.be	adare1	gQ4@.*0e+8"((r5Ydb\$ue1\$Hfx	2023-01-03	true
rgozney2@thetimes.co.uk	ldudding2	bH8\$?3b#3)*!gb`CD%	2023-10-02	false
cmeineck3@so-net.ne.jp	lgilbanks3	js0?#T A0M'mwyL(*?GG@	2023-02-22	true
loverlow4@diigo.com	eboyton4	qI0"VL~IX?MT+In8a/+0ElY7(?)	2023-02-19	true
pcranmor5@elegantthemes.com	bpedrollo5	oG0(B`Tors=a{)K,h13"'b	2023-07-12	false
hfigurski6@blogspot.com	bpapaccio6	kA8\r*vg}Z`6D@Y\S.Hdha	2023-08-30	true
vambler7@yahoo.co.jp	mdewilde7	dH4`\Q5FPJK5xi*x'ss\wL`<FL'f	2023-06-01	true
tmcphaden8@fastcompany.com	ihonig8	wX2., &%Kak\$t=2cRsWK&WSwFb&+	2023-10-02	false
cpentland9@google.com.br	sbloy9	aK2>Ed` (qR@ef i\Kbom\l<3	2023-01-03	true

Figure 6.2: Data Generated for User Table

6.2.2. Python Script: Random Values Generator

Utilizing Python scripts, the bridge tables were populated with accurate SQL notation. This approach significantly expedited the process, allowing the generation of numerous values for various table types, even those where foreign keys serve as primary keys (with automatic elimination of duplications for tables like 'Song_Playlist'). Below is an illustration of a script designed to generate random values for the 'User_Song' table (which records instances where a user tags a song as a favorite):

```
from datetime import date, timedelta
import random

songs = [34253, 12894, 56873, 49281, 27384, 18642, 56291, 41039, 23418, 65533,
         32676, 52291, 41834, 15986, 43821, 27745, 54127, 63175, 48219, 19768,
         57341, 37846, 44679, 21435, 48256]

emails = ['vhammel0@wordpress.org', 'psazio1@google.ca', 'rmarriot2@amazon.com',
          'selias3@nationalgeographic.com', 'mturneux4@desdev.cn',
          'gmaccumeskey5@weebly.com', 'semerine6@tripadvisor.com',
          'kbenmore7@cam.ac.uk', 'rplume8@tumblr.com', 'iwatkin9@bbc.co.uk',
          'ecayeta@stanford.edu', 'igerrietsb@meetup.com', 'rjillisc@irs.gov',
          'jlongstaffd@gizmodo.com', 'klednore@loc.gov', 'tlummusf@rambler.ru',
          'pdorbong@bandcamp.com', 'mstaubynh@vimeo.com', 'dofeeneyi@ustream.tv',
          'rmcelhinneyj@buzzfeed.com', 'rvanyarkink@biglobe.ne.jp',
          'ophillipsl@washingtonpost.com', 'jeastbrookm@webnode.com',
          'emacconaghyn@fastcompany.com', 'nsurpliceo@uol.com.br']

start_date, end_date = date(2022, 11, 9), date(2023, 11, 9)
set_values = set()

for _ in range(128):
    random_mail = random.choice(emails)
    random_song = random.choice(songs)
    random_date = start_date + timedelta(days=random.randint(0, (end_date - start_date).days))
    final_string = f"({random_mail}, {random_song}, '{random_date.strftime('%Y-%m-%d')}'),"
    set_values.add(final_string)

file_name = 'rnd.txt'
with open(file_name, 'w') as file:
    file.write("INSERT INTO `User_Song` (`Email`, `Content_ID`, `TagDate`) VALUES\n")
    file.write('\n'.join(set_values))

print(f"The list has been written to {file_name}")
```

In this script, 25 rows from 'User' and 'Song' tables were taken and put them into simple lists; Then a beginning date and an ending date are chosen. The values are added into a set to avoid duplicates. A for loop is used to generate a certain number of random values. Then, these values are added into a .txt file to simplify the copy and paste to MySQL.

6.2.3. Table Data Snapshots

Below, one can observe samples of filled tables. For simplicity, 25 tuples are presented in the format of a MySQL query to enhance the visualization of the data:

User Data

	Email	Username	Password	RegistrationDate	IsPremium
	abulman5@shinystat.com	lcrewe5	iA7/+8RJPQsyJ&"	2023-10-18	1
	awaterstoneb@disqus.com	wwhitnallb	pB7~5J,e#_}p&8uN	2023-05-30	1
	awithamsv@google.com.hk	lmalamorev	yI3}vBBHM4\$zWqgyHuym\$3jmW(	2023-04-09	1
	cchastandy@state.tx.us	msalzburgy	nL5_.)NNK81+rCF%&ow3	2023-06-22	0
	cpealingt@state.tx.us	lcordelett	mJ79=RQW%)qmwsq	2023-10-17	1
	cpetrus13@squidoo.com	mdurrett13	bH0(=vRG2 Ls5gO%s	2022-12-14	0
	dofeeneyi@ustream.tv	cmarneri	zI8{An<WWQ6J/,rFX@eRADn	2023-06-09	0
	ecayeta@stanford.edu	dpeirpointa	eL4,,C\$?Eto)jC#t5	2023-05-29	0
	egergec@flickr.com	nmcfaellc	kN9!<I(u96 yh(	2023-05-19	1
	emaconaghyn@fastcompany.com	cmacakn	fW8 I_H)L=t5D{~CYZ'2lX,Cx	2023-06-11	0
	ezimek9@histats.com	imulholland9	gI5!G1zeV4s@euNYu#D4y8Hl	2022-12-22	1
	gmaccumeskey5@weebly.com	jgregr5	fI5(BWcLt8zD(VqX(g'%CmW7{6	2023-06-19	1
	gpridgeonq@fda.gov	rcabbellq	wI8,e>qH0oM8KcjmUhV= 9ux	2023-09-01	1
	gshoulderu@cocolog-nifty.com	rgustusu	gC6#JcCCjv3?O.\$N'i8,>o	2023-04-21	1
	hisaksen10@wordpress.org	kdunnett10	yW9 .i"nCE=u#YoO	2022-11-26	0
	igerrietsb@meetup.com	mhenningtonb	uT0,i+7CE/zD#byIy)	2023-04-28	1
	iwatkin9@bbc.co.uk	diczokvitz9	rV4}_YV2`E6Uq9b7d,9>MT	2023-08-07	1
	jeastbrookm@webnode.com	bpurrierm	dH1&%tP3t#5J_ea`I_xGh{aEh	2023-03-30	0
	jlongstaffd@gizmodo.com	ekitneyd	iP8<rMwF}iEkOaao912!,bps	2023-06-10	1
	kandriesd@uol.com.br	heberstd	vW1#{KB?,0O>EL	2023-05-07	1
	kbenmore7@cam.ac.uk	jhethron7	xK0#2/*U>h6Xl%c%X	2023-04-25	0
	kgiovannetti11@drupal.org	wbruneton11	hM3*q_0}5W<#VS\$	2023-06-04	1
	klednore@loc.gov	fitzhakie	dY2~FWgsQ2d#_3Qpm	2023-03-15	1
	kthorington1@comsenz.com	jlangston1	xV8>dXo(F*U4>K`	2022-12-09	1
	lbarbricka@bigcartel.com	lfahya	xD4(?Uh`vv\$Y+Iv%>VQbHG4j	2022-12-13	1

Category Data

	Name	Description
	Alternative	
	Ambient	Ambient music is characterized by its atmosphe...
	Bluegrass	Bluegrass is a style of country music known for i...
	Blues	Blues music is rooted in African-American traditi...
	Blues Rock	Blues rock combines the blues' emotional depth ...
	Bossa Nova	Bossa Nova is a Brazilian music style that blends...
	Classical	Classical music is known for its complex composi...
	Country	Country music often features acoustic instrume...
	Disco	A genre of dance music containing elements of f...
	Dubstep	Dubstep is an electronic music subgenre charact...
	Electronic	Electronic music relies on synthesizers and comp...
	Flamenco	Flamenco is a passionate and expressive music ...
	Folk	Folk music is deeply rooted in cultural traditions...
	Funk	Funk is known for its groovy basslines, tight dru...
	Gospel	Gospel music is rooted in religious themes and f...
	Grunge	Grunge emerged in the 1990s and is known for i...
	Hip-Hop	Hip-hop is a genre that combines rap vocals wit...
	Indie	Indie music is often associated with independe...
	J-Pop	J-Pop is the pop music of Japan, known for its c...
	Jazz	Jazz is known for its improvisation, intricate rhy...
	K-Pop	K-Pop is a global phenomenon originating in Sou...
	Latin	
	Metal	Heavy metal features distorted guitars, powerf...
	New Wave	New Wave is a genre known for its use of synth...
	Pop	Pop music is characterized by catchy melodies, ...

Song Data

	Content_ID	Title	Duration	PublicationDate	Artist	Text	Name
	1	Thinking Out Loud	00:03:57	2014-09-24	Ed Sheeran	When your legs don't work like they used to bef...	Pop
	11245	Rolling in the Deep (Cover)	00:07:35	2010-12-14	Linkin Park	There's a fire starting in my heart...	Rock
	11269	Billie Jean (Cover)	00:02:16	2018-09-19	The Civil Wars	She was more like a beauty queen from a movie...	Folk
	12894	Bohemian Rhapsody	00:05:40	1975-10-31	Queen	Is this the real life? Is this just fantasy?	Rock
	15986	Blackbird	00:02:00	1968-11-22	The Beatles	Blackbird singing in the dead of night...	Pop
	17763	Someone You Loved	00:06:25	2019-11-08	Lewis Capaldi	Now the day bleeds into nightfall...	Pop
	17892	Imagine (Cover)	00:03:54	2020-05-08	Ariana Grande	Imagine there's no heaven...	Pop
	18642	Hotel California	00:04:54	1976-02-26	Eagles	On a dark desert highway, cool wind in my hair...	Rock
	18842	Perfect	00:04:00	2017-03-03	Ed Sheeran	I found a love for me...	Pop
	19723	Shape of You (Cover)	00:05:13	2017-07-05	Walk off the Earth	The club isn't the best place to find a lover...	Pop
	19768	Roar	00:01:55	2013-08-10	Katy Perry	I used to bite my tongue and hold my breath...	Pop
	20648	Hey Jude (Cover)	00:05:06	2014-08-08	Paul McCartney	Hey, Jude, don't make it bad...	Pop
	21435	Radio Ga Ga	00:02:35	1984-01-23	Queen	I'd sit alone and watch your light...	Rock
	23418	Hey Jude	00:04:34	1968-08-26	The Beatles	Hey, Jude, don't make it bad...	Pop
	26598	Purple Rain	00:06:45	1984-04-23	Prince	I never meant to cause you any sorrow...	Rock
	27349	Shape of You	00:03:32	2017-01-06	Ed Sheeran	The club isn't the best place to find a lover...	Pop
	27384	Billie Jean	00:04:00	1983-01-02	Michael Jackson	She was more like a beauty queen from a movie...	Pop
	27745	Wonderwall	00:05:01	1995-10-30	Oasis	Today is gonna be the day...	Rock
	28957	Take Me to Church	00:03:55	2013-09-16	Hozier	My lover's got humor, she's the giggle at a fune...	Alter...
	29435	Don't Stop Believin'	00:04:15	1981-06-03	Journey	Just a small-town girl, living in a lonely world...	Rock
	32468	Here Comes the Sun (Cover)	00:02:54	2017-10-02	Sheryl Crow	Here comes the sun, doo-doo-doo-doo...	Rock
	32584	Purple Rain (Cover)	00:03:10	2016-06-25	Beyoncé	I never meant to cause you any sorrow...	R&B
	32676	My Way	00:05:34	1969-03-02	Frank Sinatra	And now, the end is near...	Jazz
	34253	Imagine	00:03:00	1971-09-09	John Lennon	Imagine there's no heaven...	Rock
	34629	Bohemian Rhapsody (Cover)	00:03:52	2014-07-01	Pentatonix	Is this the real life? Is this just fantasy?	Pop

Podcast Data

	Content_ID	Title	Duration	PublicationDate	Author
	16589	Dateline NBC	01:00:00	2023-03-25	NBC News
	19486	The Ben Shapiro Show	00:40:00	2023-03-19	The Daily Wire
	19642	SmartLess	00:50:00	2023-01-19	Jason Bateman, Sean Hayes, Will Arnett
	26189	Hard Feelings with Jennette McCurdy	00:45:00	2023-01-15	Lemonada Media
	29376	Hip Hop Horror Stories	00:35:00	2023-02-20	iHeartPodcasts and Audio UP, Inc.
	31284	Burden of Guilt	00:30:00	2023-01-02	iHeartPodcasts and Glass Podcasts
	33457	The Kids of Rutherford County	00:55:00	2023-03-03	Serial Productions & The New York Times
	33541	Huberman Lab	00:35:00	2023-02-24	Scicomm Media
	36492	Conan O'Brien Needs A Friend	00:55:00	2023-02-18	Team Coco & Earwolf
	36872	MrBallen's Medical Mysteries	01:00:00	2023-03-16	Wondery
	37429	20/20	01:00:00	2023-03-10	ABC News
	38642	Very Scary People	00:55:00	2023-02-08	ID
	39854	Morbid	00:45:00	2023-04-06	Morbid Network Wondery
	39972	The Mel Robbins Podcast	00:30:00	2023-03-24	Mel Robbins
	41276	The Daily	00:30:00	2023-01-22	The New York Times
	42756	Wait Wait... Don't Tell Me!	00:50:00	2023-01-26	NPR
	42946	Fresh Air	00:45:00	2023-03-28	NPR
	42983	Symptomatic: A Medical Mystery Pod...	00:55:00	2023-02-23	iHeartPodcasts
	43729	Crime Junkie	00:45:00	2023-04-01	audiochuck
	48569	Heavyweight	00:30:00	2023-01-29	Spotify Studios
	52148	Up First	00:55:00	2023-02-17	NPR
	52193	Unrestorable	00:40:00	2023-03-05	iHeartPodcasts
	53897	Ghost Story	00:50:00	2023-02-09	Wondery Pineapple Street Studios
	53948	Morrison Mysteries	00:40:00	2023-03-13	NBC News
	54691	The Ezra Klein Show	01:00:00	2023-03-01	New York Times Opinion

Playlist Data

	Playlist_ID	Title	CreationDate	IsPublic	Email
	10785	Meditation Mix	2023-04-04	1	tlummusf@rambler.ru
	12493	Chill Vibes	2023-03-21	0	selias3@nationalgeographic.com
	12784	Country Roads	2023-10-05	1	gpridgeonq@fda.gov
	13972	Hip-Hop Legends	2023-08-18	1	ezimek9@histats.com
	14656	Electro Swing	2023-06-05	1	psazio1@google.ca
	16827	Metal Mayhem	2023-03-04	1	gmaccumeskey5@weebly.com
	17642	Dance Fever	2023-06-28	0	kthorrrington1@comsenz.com
	19129	Rock Ballads	2023-10-15	0	psazio1@google.ca
	20087	Funky Grooves	2023-03-30	1	mdowallz@unblog.fr
	20617	Classical Masterpieces	2023-08-03	0	dofeeneyi@ustream.tv
	21785	Indie Rock Revolution	2023-11-25	1	tlummusf@rambler.ru
	24286	R&B Soulmates	2023-11-02	0	rmarriot2@amazon.com
	24765	Rap Anthems	2023-07-18	1	ecayeta@stanford.edu
	26619	Alternative Gems	2023-04-12	0	rmarriot2@amazon.com
	27561	World Beats	2023-10-20	1	kthorrrington1@comsenz.com
	28007	Country Classics	2023-11-30	0	psazio1@google.ca
	29243	Chillwave Bliss	2023-10-10	0	psazio1@google.ca
	31491	Folk & Blues	2023-05-14	0	jlongstaffd@gizmodo.com
	31548	90s Hits	2023-04-30	0	rmarriot2@amazon.com
	32934	Guitar Solos	2023-05-31	0	kgiovannetti11@drupal.org
	35921	Smooth Jazz	2023-08-14	1	lfearnoughx@livejournal.com
	37853	Country Love Songs	2023-01-12	0	psazio1@google.ca
	37901	Reggae Rhythms	2023-06-20	0	gmaccumeskey5@weebly.com
	38500	Guitar Heroes	2023-02-10	1	kthorrrington1@comsenz.com
	39875	Mellow Acoustics	2023-05-12	0	smattamp@upenn.edu

UserListensToSong Data

	Email	Content_ID	When	Whole
	emacconaghyn@fastcompany.com	56892	2023-05-10 16:03:37	0
	awithamsv@google.com.hk	48236	2023-01-02 00:02:05	0
	rjillisc@irs.gov	20648	2023-04-29 04:26:19	0
	pdorbong@bandcamp.com	65533	2023-01-22 20:25:20	0
	lfearnoughx@livejournal.com	34912	2023-10-07 02:40:44	1
	abulman5@shinystat.com	56892	2023-04-09 20:08:50	1
	mbarrington6@comcast.net	49135	2023-03-29 18:52:50	0
	nsurpliceo@uol.com.br	11245	2023-04-25 09:41:11	1
	sleatherlandf@pcworld.com	17892	2023-05-25 09:41:50	0
	kgiovannetti11@drupal.org	44138	2023-01-03 09:24:31	0
	jeastbrookm@webnode.com	48219	2023-06-18 12:55:41	1
	ezimek9@histats.com	48219	2022-11-16 22:16:52	0
	mturneux4@desdev.cn	41592	2023-04-26 21:32:10	0
	rjillisc@irs.gov	27349	2023-03-22 00:18:34	0
	rmarriot2@amazon.com	36985	2022-11-24 03:21:05	1
	kandriesd@uol.com.br	56291	2023-08-07 14:48:01	1
	dofeeneyi@ustream.tv	37846	2023-08-14 05:55:19	1
	vhammel0@wordpress.org	57892	2023-06-13 04:29:38	0
	cpealingt@state.tx.us	23418	2023-03-21 02:46:29	0
	lbarbricka@bigcartel.com	34629	2023-05-08 16:31:54	1
	kthorrington1@comsenz.com	56291	2023-01-07 03:23:34	0
	psazio1@google.ca	41592	2022-12-21 20:46:40	1
	cpealingt@state.tx.us	34253	2023-04-10 08:10:27	1
	kbenmore7@cam.ac.uk	56873	2023-02-04 09:05:04	0
	abulman5@shinystat.com	53641	2023-09-15 12:32:59	0

UserListensToPodcast Data

	Email	Content_ID	When	Whole
	rmarriot2@amazon.com	33457	2023-02-04 06:27:42	0
	rdecents@cyberchimps.com	64785	2023-10-05 04:44:40	1
	ezimek9@histats.com	48569	2023-05-03 11:52:48	0
	cpetrus13@squidoo.com	54691	2023-09-20 14:28:03	1
	rvanyarkink@biglobe.ne.jp	39972	2023-09-13 08:32:28	1
	hisaksen10@wordpress.org	42756	2023-05-11 07:26:53	0
	gmaccumeskey5@weebly.com	36492	2023-02-05 02:16:15	1
	mmationr@hud.gov	42983	2023-09-12 07:59:34	0
	kthorrington1@comsenz.com	53897	2023-10-19 08:59:25	1
	mdowallz@unblog.fr	16589	2023-07-16 14:39:09	0
	shelsby3@sitometer.com	43729	2022-12-05 17:38:32	1
	klednore@loc.gov	38642	2022-11-27 11:22:20	0
	gshoulderu@cocolog-nifty.com	36872	2023-04-26 21:37:16	1
	egergec@flickr.com	37429	2023-10-21 10:09:20	1
	hisaksen10@wordpress.org	53948	2023-10-10 17:55:49	1
	kgiovannetti11@drupal.org	63872	2023-10-19 02:17:45	1
	kandriesd@uol.com.br	62984	2023-05-04 23:22:02	1
	rmcelhinneyj@buzzfeed.com	37429	2023-08-27 03:36:33	0
	vhammel0@wordpress.org	65192	2023-10-22 08:06:55	1
	vhammel0@wordpress.org	52148	2023-06-02 23:41:28	0
	smattamp@upenn.edu	62984	2023-10-18 00:46:49	1
	cpetrus13@squidoo.com	36492	2023-01-29 17:16:36	1
	rmarriot2@amazon.com	19642	2023-11-02 12:17:53	1
	rdecents@cyberchimps.com	42983	2023-07-16 01:50:20	0
	kthorrington1@comsenz.com	36872	2023-03-11 18:06:10	1

Song_Playlist Data

	Content_ID	Playlist_ID	InsertionDate
	17763	63388	2023-06-14
	32584	49834	2023-09-30
	56892	52913	2023-08-20
	56873	32934	2023-02-07
	62971	52913	2023-10-24
	29435	52398	2023-01-22
	54127	59348	2023-03-02
	52487	12784	2023-03-01
	48236	55060	2023-03-07
	56482	41183	2023-05-17
	63175	31491	2023-06-11
	34253	41897	2022-11-11
	45928	59348	2023-01-10
	20648	41897	2023-01-14
	43821	41897	2023-06-28
	17892	31548	2023-02-25
	21435	38500	2023-06-27
	48236	49834	2022-12-28
	19723	49834	2023-05-11
	11269	49834	2023-06-28
	61497	41183	2023-11-09
	55891	60379	2022-11-24
	45928	28007	2023-07-29
	45592	20087	2023-05-02
	48562	27561	2022-11-30

User_Song Data

	Email	Content_ID	TagDate
	dofeeneyi@ustream.tv	55782	2022-12-29
	pdorbong@bandcamp.com	43821	2023-03-24
	sleatherlandf@pcworld.com	61497	2023-11-09
	kgiovannetti11@drupal.org	34912	2023-09-08
	hisaksen10@wordpress.org	56291	2023-06-24
	mstaubynh@vimeo.com	63175	2023-05-17
	vhammel0@wordpress.org	48219	2023-10-07
	sleatherlandf@pcworld.com	21435	2023-04-27
	mstaubynh@vimeo.com	27745	2023-09-04
	cchastandy@state.tx.us	62971	2023-08-05
	rdecents@cyberchimps.com	28957	2023-07-17
	rjillisc@irs.gov	55891	2023-01-10
	semerine6@tripadvisor.com	55891	2023-02-12
	abulman5@shinystat.com	44679	2023-03-24
	rmcelhinneyj@buzzfeed.com	49135	2022-12-13
	eergec@flickr.com	17892	2023-05-02
	kbenmore7@cam.ac.uk	41834	2023-01-25
	selias3@nationalgeographi...	63921	2023-10-16
	iwatkin9@bbc.co.uk	43821	2022-11-23
	hisaksen10@wordpress.org	34629	2023-09-05
	jlongstaffd@gizmodo.com	49281	2023-10-21
	gpridgeonq@fda.gov	64198	2023-03-13
	gpridgeonq@fda.gov	17892	2022-12-30
	ezimek9@histats.com	52843	2023-04-10
	lbarbricka@bigcartel.com	64198	2023-09-04

User_Playlist Data

	Email	Playlist_ID	TagDate
	emacconaghyn@fastcompany.com	49834	2023-08-25
	vhammel0@wordpress.org	64109	2023-02-01
	gmaccumeskey5@weebly.com	55060	2023-05-19
	cpetrus13@squidoo.com	55852	2023-08-26
	mstaubynh@vimeo.com	10785	2023-03-06
	mbarrington6@comcast.net	16827	2022-11-27
	ecayeta@stanford.edu	13972	2023-07-24
	kandriesd@uol.com.br	26619	2022-12-02
	emacconaghyn@fastcompany.com	60248	2023-07-25
	awaterstoneb@disqus.com	37853	2023-10-29
	semerine6@tripadvisor.com	16827	2022-11-13
	ezimek9@histats.com	49062	2022-12-29
	smattamp@upenn.edu	31548	2023-06-04
	jlongstaffd@gizmodo.com	35921	2023-09-11
	rplume8@tumblr.com	31491	2023-08-19
	kandriesd@uol.com.br	59814	2023-02-21
	mturneux4@desdev.cn	48213	2023-09-17
	kthorington1@comsenz.com	61034	2023-07-10
	tlummusf@rambler.ru	13972	2023-11-03
	ezimek9@histats.com	31548	2023-02-18
	nsurpliceo@uol.com.br	32934	2023-03-21
	klednore@loc.gov	24765	2023-03-23
	hisaksen10@wordpress.org	58241	2023-04-03
	mstaubynh@vimeo.com	52913	2023-02-14
	lfearnoughx@livejournal.com	27561	2023-08-07

6.3. SQL3 - Performing the Queries

A comprehensive display of 21 carefully constructed queries has been built in this specific part of the project to demonstrate the robust and effective performance of the database. Each query demonstrates the system's adaptability and reactivity when carrying out a wide range of data retrieval tasks. This section provides a comprehensive overview of the database's performance, emphasising its reliability in handling different kinds of requests.

6.3.1. "WHERE" QUERIES

Query #1: Extract the user with semerine6@tripadvisor.com as email

```
SELECT *  
FROM User  
WHERE email = 'semerine6@tripadvisor.com';
```

	Email	Username	Password	RegistrationDate	IsPremium
▶	semerine6@tripadvisor.com	rgonnin6	pU3_0)qH*G"Ccv2phxbVMW*	2023-08-19	1

This query retrieves all columns from the User table for the user with the email 'semerine6@tripadvisor.com'. It's a targeted query to fetch specific user details based on their unique email identifier.

Query #2: Extract all the users that have the premium subscription

```
SELECT *  
FROM User  
WHERE IsPremium IS true;
```

Email	Username	Password	RegistrationDate	IsPremium
cpealingt@state.tx.us	lcordelett	mJ79=RQW%)qmwsg	2023-10-17	1
egergec@flickr.com	nmcfaellc	kN9!<I(u96 yh{	2023-05-19	1
ezimek9@histats.com	imulholland9	gI5!G1zeV4s@euNYu#D4y8Hl	2022-12-22	1
gmaccumeskey5@weebly.com	jgregr5	fl5(BWclt8zD(VqX(g'%CmW7{6	2023-06-19	1
gpridgeonq@fda.gov	rcabbellq	wI8,e>qH0oM8KcjmUhV=[9ux	2023-09-01	1
gshoulderu@cocolog-nifty.com	rgustusu	gC6#JeCCjv3?O.\$N'i8,>o	2023-04-21	1
igerrietsb@meetup.com	mhenningtonb	uT0,i+7CE/zD#byIy)	2023-04-28	1
iwatkin9@bbc.co.uk	diczokvitz9	rV4}_YV2`E6Uq9b7d,9>MT	2023-08-07	1
jlongstaffd@gizmodo.com	ekitneyd	iP8<rMwF)iEkOaao912!,bps	2023-06-10	1
kandriesd@uol.com.br	heberstd	vW1#(KB?,0O>EL	2023-05-07	1
kgiovannetti11@drupal.org	wbruneton11	hM3*q_0}5W<#VS\$	2023-06-04	1
klednore@loc.gov	fitzhakie	dY2~FWgsQ2d#_3Qpm	2023-03-15	1
kthorington1@comsenz.com	jlangston1	xV8>dXo(F*U4>K`	2022-12-09	1
lbarbricka@bigcartel.com	lfahya	xD4(7Uh`vv\$Y+Iv%>VQbHG4j	2022-12-13	1
lfearnyoughx@livejournal.com	kbartusekx	uT7>#`Y5KGS8kx	2023-07-14	1
mturneux4@desdev.cn	rnowell4	vS7+8eJvzCKaU<_4=Zee	2023-01-09	1
nsurpliceo@uol.com.br	vburehillo	bK4* Woo}H@z&HXp	2022-11-30	1
ophillipsl@washingtonpost.com	mmowsdill	iT4`E8m<i5BL?_s~>7B4+&F%.,	2023-01-16	1
pdorbong@bandcamp.com	cwedderburng	qS5{7^UhrMU6'7c#woL	2023-07-14	1
rdecents@cyberchimps.com	gbartoseks	dV4<DetPy(~/a~f)Yee	2023-02-27	1
rfarebrotherw@bizjournals.com	iolivierw	jS1@Qm0_(fhjpty+.0DFaC>i=,)	2023-08-06	1
rvanyarkink@biglobe.ne.jp	tpresseyk	qZ1>y%T}hEp2XTc	2023-09-27	1
semerine6@tripadvisor.com	rgonnin6	pU3_0)qH*G"Ccv2phxbVMW"	2023-08-19	1
shelsby3@sitometer.com	tgowland3	fO6)rig6n6_Tp<fy{WnmoU	2023-03-05	1
sleatherlandf@pcworld.com	qbrahmsf	dY4\$%1WG`anssILboxj@X9+th1M	2023-05-15	1

This query gathers information about users with a premium subscription by selecting all columns from the User table where the IsPremium column is true.

Query #3: Extract all the public playlists

```
SELECT *
FROM Playlist
WHERE IsPublic IS true;
```

	Playlist_ID	Title	CreationDate	IsPublic	Email
	13972	Hip-Hop Legends	2023-08-18	1	ezimek9@histats.com
	14656	Electro Swing	2023-06-05	1	psazio1@google.ca
	16827	Metal Mayhem	2023-03-04	1	gmaccumeskey5@weebly.com
	20087	Funky Grooves	2023-03-30	1	mdowallz@unblog.fr
	21785	Indie Rock Revolution	2023-11-25	1	tlummusf@rambler.ru
	24765	Rap Anthems	2023-07-18	1	ecayeta@stanford.edu
	27561	World Beats	2023-10-20	1	kthorington1@comsenz.com
	35921	Smooth Jazz	2023-08-14	1	lfearnoughx@livejournal.com
	38500	Guitar Heroes	2023-02-10	1	kthorington1@comsenz.com
	40622	Latin Grooves	2023-08-27	1	mbarrington6@comcast.net
	41897	Pop Extravaganza	2023-09-22	1	emaconaghyn@fastcompan...
	45156	Bluesy Tunes	2023-05-09	1	shelsby3@sitemeter.com
	47328	Awesome Mix	2023-02-14	1	rmarriot2@amazon.com
	49062	Soulful Sounds	2023-09-16	1	gmaccumeskey5@weebly.com
	52913	EDM Party Mix	2023-02-27	1	rjillisc@irs.gov
	52987	Pop Party Hits	2023-02-18	1	psazio1@google.ca
	55060	Salsa Sensation	2023-09-11	1	mmationr@hud.gov
	55852	Hip-Hop Hits	2023-07-29	1	rdecents@cyberchimps.com
	58241	Saxophone Serenades	2023-09-02	1	mmationr@hud.gov
	58309	Rock Classics	2023-01-09	1	rmarriot2@amazon.com
	59814	Motown Magic	2023-07-15	1	awaterstoneb@disqus.com
	60248	Indie Favorites	2023-06-15	1	ecayeta@stanford.edu
	61034	Electronic Dreams	2023-01-25	1	ezimek9@histats.com
	63388	Soulful Ballads	2023-06-10	1	rmcelhinneyj@buzzfeed.com
	63946	Classic Rock	2023-07-06	1	kthorington1@comsenz.com
	63995	80s Nostalgia	2023-03-15	1	kthorington1@comsenz.com

Here, the query focuses on playlists, extracting all columns from the Playlist table where the IsPublic column is marked as true, indicating that the playlist is public.

6.3.2. "WHERE, LIMIT, LIKE" QUERIES

Query #4: Extract the first ten songs that contain “Cover” in their title

```
SELECT *
FROM Song
WHERE Title LIKE '%Cover%'
LIMIT 10;
```

	Content_ID	Title	Duration	PublicationDate	Artist	Text	Name
	11245	Rolling in the Deep (Cover)	00:07:35	2010-12-14	Linkin Park	There's a fire starting in my heart...	Rock
	11269	Billie Jean (Cover)	00:02:16	2018-09-19	The Civil Wars	She was more like a beauty queen from a movie...	Folk
	17892	Imagine (Cover)	00:03:54	2020-05-08	Ariana Grande	Imagine there's no heaven...	Pop
	19723	Shape of You (Cover)	00:05:13	2017-07-05	Walk off the Earth	The club isn't the best place to find a lover...	Pop
	20648	Hey Jude (Cover)	00:05:06	2014-08-08	Paul McCartney	Hey, Jude, don't make it bad...	Pop
	32468	Here Comes the Sun (Cover)	00:02:54	2017-10-02	Sheryl Crow	Here comes the sun, doo-doo-doo-doo...	Rock
	32584	Purple Rain (Cover)	00:03:10	2016-06-25	Beyoncé	I never meant to cause you any sorrow...	R&B
	34629	Bohemian Rhapsody (Cover)	00:03:52	2014-07-01	Pentatonix	Is this the real life? Is this just fantasy?	Pop
	36985	Hallelujah (Cover)	00:03:59	2012-12-21	Pentatonix	Now I've heard there was a secret chord...	Pop
	45592	Hotel California (Cover)	00:04:04	2015-03-12	The Killers	On a dark desert highway, cool wind in my hair...	Rock

This query specifically targets songs with titles containing the word "Cover." It selects all columns from the Song table, utilizing the LIKE operator with the '%' wildcard for flexible pattern matching, and limits the result to the first ten records.

Query #5: Extract the email of only five users that, in their username, contain 'll'

```
SELECT email
FROM User
WHERE Username LIKE '%ll%'
LIMIT 5;
```

	email
▶	awaterstoneb@disqus.com
	egeger@flickr.com
	ezimek9@histats.com
	gpridgeonq@fda.gov
	mturneux4@desdev.cn

This query retrieves the email addresses of users whose usernames contain the substring "ll." It selects the email column from the User table, utilizing the LIKE operator with the '%' wildcard for pattern matching, and limits the result to the first five records.

Query #6: Extract three songs that contain 'You' or 'Love' in their title

```
SELECT *
FROM Song
WHERE Title LIKE '%You%' OR '%Love%'
LIMIT 3;
```

	Content_ID	Title	Duration	PublicationDate	Artist	Text	Name
▶	17763	Someone You Loved	00:06:25	2019-11-08	Lewis Capaldi	Now the day bleeds into nightfall...	Pop
	19723	Shape of You (Cover)	00:05:13	2017-07-05	Walk off the Earth	The club isn't the best place to find a lover...	Pop
	27349	Shape of You	00:03:32	2017-01-06	Ed Sheeran	The club isn't the best place to find a lover...	Pop

In this query, the focus is on songs with titles containing either "You" or "Love." It selects all columns from the Song table, using the LIKE operator with the '%' wildcard for flexibility, and limits the result to the first three records.

6.3.3. "WHERE, IN, Nested Query" QUERIES

Query #7: Extract the Title of the playlists created by users that are not Premium

```
SELECT Title
FROM Playlist
WHERE email IN (SELECT email
                FROM User
                WHERE IsPremium IS false);
```

	Title
▶	Meditation Mix
	Chill Vibes
	Electro Swing
	Rock Ballads
	Funky Grooves
	Classical Masterpieces
	Indie Rock Revolution
	R&B Soulmates
	Rap Anthems
	Alternative Gems
	Country Classics
	Chillwave Bliss
	90s Hits
	Country Love Songs
	Mellow Acoustics
	Latin Grooves
	Pop Extravaganza
	Awesome Mix
	Inspirational Hits
	EDM Party Mix
	Pop Party Hits
	Salsa Sensation
	Saxophone Serenades
	Rock Classics
	Indie Favorites
	Hard Rock Anthems
	Golden Oldies

This query extracts the titles of playlists created by non-premium users. It uses a subquery to identify non-premium users' emails and then filters playlists based on these emails, selecting only the titles from the Playlist table.

Query #8: Extract the Title of the songs that are in all playlists created by the user with 'psazio1@google.ca' as email

```
SELECT s.Title
FROM Song s
WHERE s.Content_ID IN (SELECT sp.Content_ID
                       FROM Song_Playlist sp
                       WHERE s.Content_ID = sp.Content_ID
                       AND sp.Playlist_ID IN (SELECT p.Playlist_ID
                                             FROM Playlist p
                                             WHERE sp.Playlist_ID = p.Playlist_ID AND p.Email = 'psazio1@google.ca'));
```

	Title
▶	Wonderwall
	Radioactive
	Rocket Man
	What a Wonderful World
	Here Comes the Sun
	All You Need Is Love
	Bohemian Rhapsody
	Don't Stop Believin'
	My Way
	Blackbird (Cover)
	Blackbird
	Bohemian Rhapsody (C...
	Purple Haze
	What's Going On
	Superstition
	Thriller
	Uptown Funk
	Yesterday (Cover)
	All You Need Is Love (C...
	Roar
	Hey Jude
	Shape of You
	Take Me to Church
	Imagine
	Sweet Child o' Mine (Co...
	Stairway to Heaven
	Side Street Walk to Home

This query targets songs that are part of all playlists created by a specific user ('psazio1@google.ca'). It uses nested subqueries to filter songs based on content ID, ensuring they belong to playlists associated with the specified user email.

6.3.4. "GROUP BY, 1 JOIN, AS" QUERIES

Query #9: Count the number of songs in each category and display it along with the category name

```
SELECT c.NAME AS CategoryName, COUNT(s.CONTENT_ID) AS SongCount
FROM Category c
LEFT JOIN Song s ON c.NAME = s.NAME
GROUP BY c.NAME;
```

CategoryName	SongCount
Country	1
Disco	0
Dubstep	0
Electronic	0
Flamenco	0
Folk	4
Funk	1
Gospel	0
Grunge	1
Hip-Hop	0
Indie	0
J-Pop	0
Jazz	2
K-Pop	0
Latin	1
Metal	0
New Wave	0
Pop	27
Psychedelic	0
Punk	0
R&B	3
Raggae	0
Rock	21
Salsa	0
Ska	0
Soul	2
World Music	0

In this query, the goal is to count the number of songs in each category. It performs a left join between the Category and Song tables, ensuring all categories are considered, even those without songs. The result is grouped by category name, providing a comprehensive count.

Query #10: For each podcast, count the number of DIFFERENT users who listened to it at least once

```
SELECT p.Title, p.Content_ID, COUNT(DISTINCT email) AS DistinctUserCount
FROM UserListensToPodcast AS u
JOIN Podcast AS p ON u.Content_ID = p.Content_ID
GROUP BY p.Title, p.Content_ID;
```

	Title	Content_ID	DistinctUserCount
►	20/20	37429	21
	48 Hours	57823	16
	Burden of Guilt	31284	15
	Conan O'Brien Needs A Friend	36492	9
	Crime Junkie	43729	14
	Dateline NBC	16589	14
	Fresh Air	42946	13
	Ghost Story	53897	11
	Hard Feelings with Jennette McCurdy	26189	15
	Heavyweight	48569	18
	Hidden Brain	65192	12
	Hip Hop Horror Stories	29376	11
	Huberman Lab	33541	16
	JoJo Siwa Now	62984	13
	Morbid	39854	14
	Morrison Mysteries	53948	15
	MrBallen's Medical Mysteries	36872	17
	Murder in Apartment 12	63872	19
	My Favorite Murder with Karen Kilga...	55217	6
	New Heights with Jason and Travis K...	55683	10
	SmartLess	19642	13
	Stuff You Should Know	55986	13
	Symptomatic: A Medical Mystery Pod...	42983	18
	The Ben Shapiro Show	19486	11
	The Daily	41276	13
	The Ezra Klein Show	54691	12

This query focuses on podcasts, counting the number of distinct users who listened to each podcast at least once. It utilizes a join between the UserListensToPodcast and Podcast tables, grouping the results by podcast title and content ID, and counting distinct user emails.

6.3.5. "WHERE, GROUP BY" QUERIES

Query #11: Count the number of favourite songs for each user during March 2023”

```
SELECT email, COUNT(*) AS NumFavouriteSongs
FROM User_Song
WHERE TagDate BETWEEN '2023-03-01' AND '2023-03-31'
GROUP BY email;
```

	email	NumFavouriteSongs
▶	abulman5@shinystat.com	1
	awithamsv@google.com.hk	1
	cchastandy@state.tx.us	3
	cpealingt@state.tx.us	1
	cpetrus13@squidoo.com	2
	dofeeneyi@ustream.tv	1
	ecayeta@stanford.edu	1
	egergec@flickr.com	2
	emaconaghyn@fastcompany.com	1
	ezimek9@histats.com	3
	gmaccumeskey5@weebly.com	1
	gpridgeonq@fda.gov	2
	igerrietsb@meetup.com	1
	iwatkin9@bbc.co.uk	2
	jeastbrookm@webnode.com	1
	kandriesd@uol.com.br	2
	kgiovannetti11@drupal.org	1
	kthorington1@comsenz.com	1
	lbarbricka@bigcartel.com	2
	mbarrington6@comcast.net	1
	mmationr@hud.gov	3
	pdorbong@bandcamp.com	2
	psazio1@google.ca	1
	rdecents@cyberchimps.com	1
	rjillisc@irs.gov	1
	rmcelhinneyj@buzzfeed.com	2
	rdmcc@bible.com	1

This query calculates the number of favorite songs tagged by each user specifically during March 2023. It filters the results based on the TagDate and uses the GROUP BY clause to organize the counts by user email.

Query #12: Extract the average duration of all songs that belong to 'Pop' music category

```
SELECT s.NAME AS Category, AVG(s.Duration) AS AvgDuration
FROM Song s
WHERE s.NAME = 'Pop'
GROUP BY s.NAME;
```

	Category	AvgDuration
▶	Pop	368.9259

This query calculates the average duration of songs belonging to the 'Pop' music category. It filters the results based on the category name and uses the GROUP BY clause to present the average duration for each category.

6.3.6. "GROUP BY, HAVING, AS" QUERIES

Query #13: Extract the email of users who tagged at least 10 songs as favourite, in descending order

```
SELECT Email, COUNT(*) AS NumFavouriteSongs
FROM UserListensToSong
GROUP BY Email
HAVING NumFavouriteSongs >= 10
ORDER BY NumFavouriteSongs DESC;
```

	Email	NumFavouriteSongs
▶	kandriesd@uol.com.br	20
	rmarriot2@amazon.com	19
	pdorbong@bandcamp.com	15
	vhammel0@wordpress.org	15
	jlongstaffd@gizmodo.com	14
	kthorington1@comsenz.com	14
	abulman5@shinystat.com	13
	awithamsv@google.com.hk	13
	iwatkin9@bbc.co.uk	13
	ophillipsl@washingtonpost.com	13
	rmcelhinneyj@buzzfeed.com	13
	gshoulderu@cocolog-nifty.com	12
	kgiovannetti11@drupal.org	12
	lbarbricka@bigcartel.com	12
	emacconaghyn@fastcompany...	11
	jeastbrookm@webnode.com	11
	lfearnoughx@livejournal.com	11
	mbarrington6@comcast.net	11
	rfarebrotherw@bizjournals.com	11
	semerine6@tripadvisor.com	11
	tlummusf@rambler.ru	11
	cpealingt@state.tx.us	10
	gpridgeonq@fda.gov	10
	igerrietsb@meetup.com	10
	mdowallz@unblog.fr	10
	mturneux4@desdev.cn	10
	...	10

This query identifies users who have tagged at least 10 songs as favorites. It uses the GROUP BY clause to organize the results by user email, applies the HAVING clause to filter based on the minimum count requirement, and orders the results in descending order of the number of favorite songs.

Query #14: Extract the email of users that created at least 3 playlists

```
SELECT Email, COUNT(*) NumPlaylistCreated
FROM Playlist
GROUP BY Email
HAVING NumPlaylistCreated > 2;
```

	Email	NumPlaylistCreated
▶	gmaccumeskey5@weebly.com	3
	kthorington1@comsenz.com	5
	mmationr@hud.gov	3
	psazio1@google.ca	6
	rmarriot2@amazon.com	6

This query retrieves the email addresses of users who have created at least 3 playlists. It uses the GROUP BY clause to organize the results by user email and applies the HAVING clause to filter based on the minimum count requirement.

6.3.7. "WHERE, GROUP BY, HAVING, AS" QUERIES

Query #15: Extract the email of users that have tagged at least two favourite songs during February 2023

```
SELECT email, COUNT(*) AS FavouriteSongs
FROM User_Song
WHERE TagDate BETWEEN '2023-02-01' AND '2023-02-28'
GROUP BY email
HAVING FavouriteSongs >= 2;
```

	email	FavouriteSongs
▶	cpealingt@state.tx.us	3
	dofeeneyi@ustream.tv	2
	emacconaghyn@fastcompany.com	2
	rdecents@cyberchimps.com	4
	rvanyarkink@biglobe.ne.jp	3

This query extracts the email addresses of users who have tagged at least two favorite songs during February 2023. It filters the results based on the TagDate, uses the GROUP BY clause to organize the counts by user email, and applies the HAVING clause to specify the minimum count requirement.

Query #16: Extract the playlist IDs that have at least 10 songs

```
SELECT Playlist_ID, COUNT(Content_ID) as NumSongs
FROM Song_Playlist
GROUP BY Playlist_ID
HAVING NumSongs >= 10;
```

	Playlist_ID	NumSongs
▶	10785	11
	12784	12
	13972	11
	16827	13
	17642	14
	20087	12
	21785	11
	24765	15
	26619	10
	28007	10
	35921	12
	37853	11
	38500	12
	39875	11
	41183	11
	41897	10
	45156	15
	47328	10
	48213	10
	49062	11
	49834	15
	52398	14
	52987	10
	55060	14
	58241	10
	58309	10
	58348	11

This query identifies playlist IDs that have at least 10 songs. It uses the GROUP BY clause to organize the results by playlist ID and applies the HAVING clause to filter based on the minimum song count.

6.3.8. "WHERE, Nested Query, GROUP BY" QUERIES

Query #17: Count the number of songs in each playlist where the playlist is marked as public, and display the result with the playlist title

```
SELECT Title, Playlist_ID, (SELECT COUNT(*)
                           FROM Song_Playlist s
                           WHERE p.Playlist_ID = s.Playlist_ID) AS '#Songs'
FROM Playlist p
WHERE IsPublic IS true
GROUP BY Title, Playlist_ID;
```

	Title	Playlist_ID	#Songs
►	Meditation Mix	10785	11
	Country Roads	12784	12
	Hip-Hop Legends	13972	11
	Electro Swing	14656	6
	Metal Mayhem	16827	13
	Funky Grooves	20087	12
	Indie Rock Revolution	21785	11
	Rap Anthems	24765	15
	World Beats	27561	5
	Smooth Jazz	35921	12
	Guitar Heroes	38500	12
	Latin Grooves	40622	5
	Pop Extravaganza	41897	10
	Bluesy Tunes	45156	15
	Awesome Mix	47328	10
	Soulful Sounds	49062	11
	EDM Party Mix	52913	6
	Pop Party Hits	52987	10
	Salsa Sensation	55060	14
	Hip-Hop Hits	55852	7
	Saxophone Serenades	58241	10
	Rock Classics	58309	10
	Motown Magic	59814	9
	Indie Favorites	60248	9
	Electronic Dreams	61034	7
	Soulful Ballads	63388	12
	Classic Rock	63846	15

This query counts the number of songs in each public playlist from the Playlist table and displays the result along with the playlist title. It utilizes a subquery to count the songs associated with each playlist and applies the GROUP BY clause for title and playlist ID.

6.3.9. "WHERE, GROUP BY, HAVING, 1 JOIN" QUERIES

Query #18: Find playlists with more than 10 songs and display their titles

```
SELECT p.Title, COUNT(sp.CONTENT_ID) AS NumSongs
FROM Playlist p
JOIN Song_Playlist sp ON p.PLAYLIST_ID = sp.PLAYLIST_ID
GROUP BY p.Title
HAVING numSongs > 10;
```

	Title	NumSongs
▶	Meditation Mix	11
	Country Roads	12
	Hip-Hop Legends	11
	Metal Mayhem	13
	Dance Fever	14
	Funky Grooves	12
	Indie Rock Revolution	11
	Rap Anthems	15
	Smooth Jazz	12
	Country Love Songs	11
	Guitar Heroes	12
	Mellow Acoustics	11
	Singer-Songwriter Sh...	11
	Bluesy Tunes	15
	Soulful Sounds	11
	Piano Classics	15
	Inspirational Hits	14
	Salsa Sensation	14
	Piano Perfection	11
	Hard Rock Anthems	13
	Soulful Ballads	12
	Classic Rock	15
	Rhythms of Africa	12
	Jazz Journeys	15

This query identifies playlists with more than 10 songs by joining the Playlist and Song_Playlist tables. It then uses the GROUP BY clause for title and applies the HAVING clause to filter out playlists with less than or equal to 10 songs, presenting a refined list of substantial playlists.

Query #19: Extract the titles of podcasts that are listened to by at least 15 users and that have a duration of at least 45 minutes

```
SELECT p.Title, p.Duration, COUNT(DISTINCT u.Email) AS UniqueListeners
FROM Podcast p
JOIN UserListensToPodcast u ON p.Content_ID = u.Content_ID
WHERE p.Duration >= '00:45:00'
GROUP BY p.Title, p.Duration
HAVING UniqueListeners >= 15;
```

	Title	Duration	UniqueListeners
▶	20/20	01:00:00	21
	48 Hours	00:45:00	16
	Hard Feelings with Jennette McCurdy	00:45:00	15
	MrBallen's Medical Mysteries	01:00:00	17
	Symptomatic: A Medical Mystery Podcast	00:55:00	18

This query retrieves the titles of podcasts that are listened to by at least 15 users and have a duration of at least 45 minutes. It joins the Podcast and UserListensToPodcast tables, filters based on duration and user count, and uses the GROUP BY clause for title and duration.

6.3.10. "WHERE, GROUP BY, HAVING, 2 JOINs" QUERIES

Query #20: Extract the title of songs that are part of at least 3 public playlists

```
SELECT s.Title, COUNT(s_p.Playlist_ID) AS InHowManyPublicPlaylists
FROM Song As s
JOIN Song_Playlist s_p ON s.Content_ID = s_p.Content_ID
JOIN Playlist p ON s_p.Playlist_ID = p.Playlist_ID
WHERE p.IsPublic IS true
GROUP BY s.Title, s.Content_ID
HAVING InHowManyPublicPlaylists >= 3;
```

	Title	InHowManyPublicPlaylists
▶	Hotel California	5
	Shape of You (Cover)	5
	Wonderwall	4
	Don't Stop Believin'	3
	Rolling in the Deep	4
	Uptown Funk	3
	Can't Help Falling in Love	3
	Livin' on a Prayer	8
	Stairway to Heaven	7
	All You Need Is Love (Cover)	6
	Girls Just Want to Have Fun	5
	Rolling in the Deep (Cover)	3
	Dancing Queen	6
	Hallelujah (Cover)	3
	Like a Rolling Stone	4
	Despacito	5
	Blackbird (Cover)	5
	Let's Get It On (Cover)	6
	Smells Like Teen Spirit (Cover)	7
	Imagine (Cover)	5
	Don't Stop Me Now	8
	Imagine (Cover)	10
	Hey Jude (Cover)	9
	Hey Jude	7
	Shape of You	3
	Take Me to Church	5
	Radioactive	5

This query extracts the titles of songs that are part of at least 3 public playlists. It joins the Song, Song_Playlist, and Playlist tables, filters based on public playlists, and uses the GROUP BY clause for title and content ID, applying the HAVING clause for the minimum playlist count.

Query #21: Extract the title of playlists that were created between March 2023 and May 2023 and that were tagged as favourites by at least 5 distinct users that DO NOT have a premium subscription

```
SELECT p.Title, COUNT(DISTINCT u.Email) AS DistinctTagFavourite
FROM Playlist AS p
JOIN User_Playlist AS u_p ON p.Playlist_ID = u_p.Playlist_ID
JOIN User u ON u_p.Email = u.Email
WHERE p.CreationDate BETWEEN '2023-03-01' AND '2023-05-31' AND u.IsPremium = false
GROUP BY p.Title
HAVING DistinctTagFavourite >= 5;
```

	Title	DistinctTagFavourite
►	Folk & Blues	5
	Guitar Solos	5
	Hard Rock Anthems	6
	Metal Mayhem	6

This query extracts the titles of playlists created between March 2023 and May 2023, tagged as favorite by at least 5 distinct users without a premium subscription. It joins the Playlist, User_Playlist, and User tables, filtering based on creation date and premium status, and uses the GROUP BY clause for title, applying the HAVING clause for the minimum user count.

7 | CONCLUSION

In summary, this project successfully addressed the challenging task of developing a robust data structure for an application providing music streaming services. Initially, an Entity-Relationship (ER) model was drawn up to visually represent the most abstract structure lying behind the entities and their interrelations.

Subsequently, the analysis proceeded with a rigorous design of a Logical Model (LM), aimed at improving the database's design structure, which eventually paved the way for the development of a Relational Model (RM), critical for effectively translating the representations provided by the previous two models into a first set of visualizable tables presenting well-defined primary and foreign keys connections.

The utilization of MySQL for the construction of the database brought the most technical component of the entire project, guaranteeing that all the previous conceptual frameworks were integrated smoothly into a fully operational database capable of storing and retrieving accurately relevant pieces of information. In order to validate the effective functioning of the proposed design, a series of example queries was run and displayed in the document each one with its respective output.

To conclude, this project accomplished the primary objective of data design modelling and construction via a methodical path from design to practical implementation, thus resulting in a coherent and functional music application database, able to promptly satisfy the requirements of keeping and accessing relevant data whenever it's needed.

